

Reflexión de la Práctica

Gestión de Proyectos con Node.js, Express y MySQL

1. ¿Cómo diseñé los endpoints?

Para diseñar los endpoints seguí el patrón REST, organizando cada ruta según el recurso que maneja y la acción que realiza. Para la tabla de proyectos definí cuatro rutas sobre `/api/proyectos`: una con GET para obtener todos los registros, otra con POST para crear uno nuevo, una con PUT usando el parámetro dinámico `/:id` para actualizar un proyecto específico, y finalmente DELETE con `/:id` para eliminarlo.

Para los responsables agregué un endpoint separado en `/api/responsables` con método GET, ya que son un recurso independiente que el formulario necesita consultar al cargar la página. Tener rutas distintas para cada recurso me permitió mantener el código organizado y fácil de entender.

Cada endpoint valida que los campos obligatorios estén presentes antes de ejecutar la consulta SQL, y en caso de error regresa un status HTTP adecuado (400 para datos faltantes, 404 si no se encuentra el recurso, 500 para errores del servidor).

2. ¿Qué problemas tuve al cargar el select?

El principal problema fue que la función `cargarResponsables()` estaba apuntando a la URL incorrecta. En lugar de hacer la petición a `/api/responsables`, estaba usando la misma constante `API_URL` que apunta a `/api/proyectos`, por lo que el select recibía la lista de proyectos en vez de responsables, y los campos nombre y puesto no existían en esos objetos.

Otro problema fue que el `forEach` que construye los elementos option estaba mezclado dentro de la función `mostrarProyectos()`, cuando debía estar en una función separada llamada `mostrarResponsables()`. Esto causaba que el select nunca se llenara correctamente porque ese bloque de código se ejecutaba en el contexto equivocado.

La solución fue crear la constante `RESPONSABLES_URL` con el endpoint correcto y mover toda la lógica de construcción del select a su propia función, limpiando las opciones anteriores antes de agregar las nuevas para evitar duplicados si la función se llamara más de una vez.

3. ¿Por qué usé id y no nombre en el select?

Usé el id del responsable como valor del option porque es la llave primaria en la tabla de responsables y es lo que la tabla de proyectos necesita guardar como referencia. Si guardara el nombre, cualquier cambio en el nombre del responsable rompería la relación entre tablas, además de que dos personas podrían tener el mismo nombre.

Al guardar el id en la columna responsable de proyectos, la base de datos puede hacer JOIN entre las dos tablas de forma confiable. El id es inmutable y único, lo que garantiza integridad referencial.

En el `option.textContent` sí uso el nombre y puesto para que el usuario vea información legible, pero el valor que viaja al servidor cuando se envía el formulario es siempre el id numérico.

4. ¿Cómo funciona el JOIN?

El INNER JOIN permite combinar filas de dos tablas en una sola consulta cuando existe una coincidencia entre una columna de cada una. En este caso la condición es `ON r.id = p.responsable`, lo que le indica a MySQL que relacione cada proyecto con el responsable cuyo id sea igual al valor guardado en la columna responsable del proyecto.

Gracias al JOIN, en lugar de hacer dos peticiones separadas (una para el proyecto y otra para buscar el responsable), obtengo todo en una sola consulta. Los campos de la tabla responsables se traen con alias como `responsable_nombre`, `responsable_correo` y `responsable_puesto` para distinguirlos de los campos de proyectos y evitar conflictos de nombres.

En el frontend, el objeto que llega por JSON ya incluye `responsable_nombre` directamente, por lo que en la tarjeta de cada proyecto solo necesito acceder a `p.responsable_nombre` para mostrarlo, sin necesidad de hacer ninguna búsqueda adicional.